

常微分方程模型

Ordinary Differential Equation Models

在经济、社会、军事、工程和自然界等领域中，有很多问题都可以看做是实际对象的某些特性随着时间或空间的变化而演变的过程，这一过程可借助微分方程来描述。**微分方程模型是通过机理分析，利用微元法找出现象的内在规律并建立瞬时变化率表达式，再根据所给的特定条件，求解微分方程，并预测现象的未来性态，控制其发展趋势。**含有未知函数的导数（或偏导数）的方程称为**微分方程**。若未知函数为一元函数，则称相应的微分方程为**常微分方程**。

饮酒驾车模型

当驾驶人血液中酒精含量达到 $80\text{mg}/100\text{ml}$ 时，发生交通事故的几率是血液中不含酒精时的2.5倍；达到 $100\text{mg}/100\text{ml}$ 时，发生交通事故的几率是血液中不含酒精时的4.7倍；即使在少量饮酒的状态下，交通事故的危险度也可达到未饮酒状态的两倍左右。

☀ 酒驾: $20 \leq \text{酒精含量} < 80\text{mg} / 100\text{ml}$

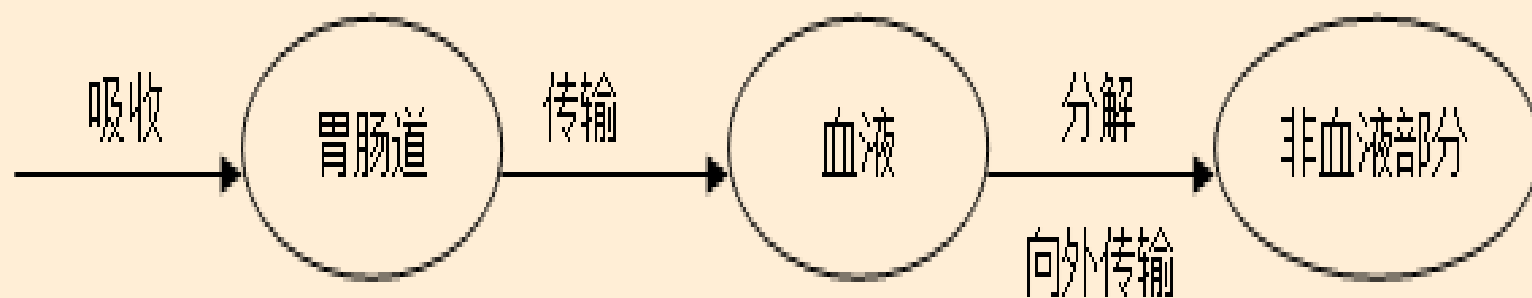
☀ 醉驾: $\text{酒精含量} \geq 80\text{mg} / 100\text{ml}$



某人在中午12点喝了一瓶啤酒，下午6点检查时符合驾车标准，紧接着他在吃晚饭时又喝了一瓶啤酒，为了保险起见，他呆到凌晨零点才驾车回家，又一次遭遇检查，却被定为饮酒驾车，这既让他懊恼又让他困惑，为什么喝同样多的酒且时间间隔差不多，检测结果会不一样呢？



机理分析：酒精由胃肠道吸收，再传输到血液，部分分解（向外传输到非血液部分），这里只考虑血液部分的酒精浓度问题，非血液部分不在讨论范围之内。酒精的传输过程如图所示。

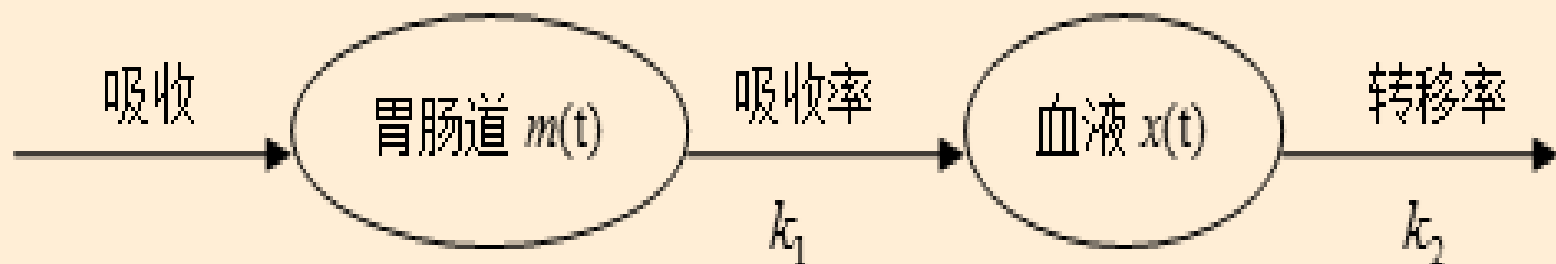


假设：（1）血液中酒精分布是均匀的；（2）酒精在胃肠道中的吸收是一级动力学过程，即胃肠道中酒精吸收率正比于肠胃道中酒精含量，比例系数为 k_1 ；（3）血液中的酒精转移率正比于血液中的酒精含量，比例系数为 k_2 ；（4）忽略从饮酒到酒精开始吸收的时间延迟；（5）由于呼吸和排尿对体液中的酒精含量影响很小，因此不考虑呼吸、尿液对酒精总量的影响。

设 t 时刻胃肠道中的酒精含量为 $m(t)$ ，血液中的酒精含量为 $x(t)$ ，考虑用微元法来分析胃肠道和血液中酒精含量的变化。

在时间段 $[t, t+\Delta t]$ 内胃肠道中的酒精含量的改变量等于被血液吸收的酒精含量，即有

$$m(t+\Delta t)-m(t) = -k_1 m(t)\Delta t$$



等式两边同除以 Δt ，并令 $\Delta t \rightarrow 0$ ，则有

$$dm/dt = -k_1 m(t)$$

初始时刻 $t=0$ ，饮酒者喝完一瓶酒，酒精迅速进入肠胃道，故可令 $m(0)=c$ ， c 为一瓶啤酒的酒精含量。

Matlab求解，程序如下：

```
dsolve('Dm+k1*m=0','m(0)=c','t') %利用dsolve求解微分方程
```

运行结果为：ans = c*exp(-k1*t)

即胃肠道中酒精含量方程为

$$m(t) = ce^{-k_1 t}$$

2. 常微分方程 (Ordinary Differential Equation)

MATLAB使用函数dsolve来求解常微分方程:

格式: `>> syms y(t) x(t) ... ; %假设t为自变量`

`>> dsolve([eq1, eq2, ...], [cond1, cond2, ...])`

其中, eq1, eq2, ...代表常微分方程式 (用`==`连接); cond1, cond2, ...为初始条件 (用`==`连接), 省略给出通解。

注意: (1) 在微分方程式的输入中, 用 `diff(y,t)`、`diff(y,t,2)` 分别表示 $y(t)$ 的一阶、二阶导数, 同时 **方程用`==`连接**;

(2) **初始条件也是用`==`连接**, 如 `y(0)==1`, `Dy(0)==1` (这里需先声明 `Dy=diff(y,t)`)。

Example 8-25 求常微分方程

1) $y'=x$ 的通解以及在初始条件 $y(0)=1$ 下的特解。

```
>>syms y(x)
```

```
>>dsolve(diff(y,x)==x)    %指定x为自变量。
```

```
ans =
```

```
x^2/2 + C1
```

```
>>dsolve(diff(y,x)==x, y(0)==1) %初始你条件为y(0)=1
```

```
ans =
```

```
x^2/2 + 1
```

2) 求微分方程 $y''=1+y'$ 当 $y(0)=1, y'(0)=0$ 的解

```
>>syms y(t);
```

```
>> Dy=diff(y,t);
```

```
>>cond=[y(0)==1, Dy(0)==0];
```

```
>>dsolve(diff(y,t,2)==1+diff(y,t),cond)
```

```
ans =
```

```
exp(t)-t
```

同理，在时间段 $[t, t+\Delta t]$ 内血液中酒精含量的改变量应当等于胃肠道中酒精的转化量减去转移为非血液部分的酒精含量，即

$$x(t + \Delta t) - x(t) = k_1 m(t) \Delta t - k_2 x(t) \Delta t$$

等式两边同除以 Δt ，并令 $\Delta t \rightarrow 0$ ，则有

$$dx/dt = k_1 m(t) - k_2 x(t)$$

将 $m(t)$ 代入，得到

$$dx/dt = k_1 c e^{-k_1 t} - k_2 x(t)$$

初始时刻 $t=0$ 时，认为血液中无酒精，即 $x(0)=0$ 。

Matlab程序为:

```
dsolve('Dx-k1*c*exp(-k1*t)+k2*x=0','x(0)=0','t')
```

运行结果为: $\text{ans} = (c*k1/(-k1+k2)*\exp(-t*(k1-k2))-c*k1/(-k1+k2))*\exp(-k2*t)$

即血液中的酒精含量方程为:

$$x(t) = ck_1/(k_2 - k_1) \cdot (e^{-k_1 t} - e^{-k_2 t})$$

若不考虑脂肪等因素对血液中酒精含量的影响，对饮酒人喝一瓶啤酒后采取血液抽样，得到下表1。

表1 血液中酒精含量（单位：mg/100ml）

时间(小时)	0.25	0.5	0.75	1	1.5	2	2.5	3	3.5	4	4.5	5
酒精含量	15	34	37.5	41	41	38.5	34	34	34	25.5	25	20.5
时间(小时)	6	7	8	9	10	11	12	13	14	15	16	
酒精含量	19	17.5	14	12.5	9	7.5	6	5	3.5	3.5	2	

为了拟合曲线 $\mathbf{x}(\mathbf{t})$ ，将 $x(t) = ck_1/(k_2 - k_1) \cdot (e^{-k_1t} - e^{-k_2t})$

改写成 $x(t) = a_1(e^{-a_2t} - e^{-a_3t})$

下面给出Matlab拟合曲线 $\mathbf{x}(\mathbf{t})$ 的程序.

先建立M文件fun1.m:

```
function y=fun1(a,x) %函数名称fun1，输入参数为a和x
```

```
y=a(1)*(exp(-a(2)*x)-exp(-a(3)*x));
```

在命令窗口运行:

```
x=[0.25 0.5 0.75 1 1.5 2 2.5 3 3.5 4 4.5 5 6 7 8 9 10  
11 12 13 14 15 16];
```

```
y=[15 34 36.5 41 41 38.5 34 34 29 25.5 25.5 20.5 19  
17.5 14 12.5 9 7.5 6 5 3.5 3.5 2];
```

```
a=lsqcurvefit('fun1',[100;0.1;2],x,y)
```

%%Matlab中调用lsqcurvefit实现普通最小二乘估计

运行结果为: $a(1)=60.0484$, $a(2)= 0.1950$,

$a(3)= 1.8537$

2、lsqcurvefit函数

用于非线性最小二乘拟合，也就是拟合一个非线性函数。

`x = lsqcurvefit(fun,x0,xdata,ydata)`

其中，`fun`是自定义的非线性函数句柄，`x0`是初始参数猜测值，`xdata`和`ydata`分别是数据点的横坐标和纵坐标向量，`x`是拟合得到的最优参数。

Example 5-18

```
fun = @(x,xdata) x(1)*exp(-x(2)*xdata); % 定义非线性函数
xdata = [3 6 9 12 15 18 21 24]; % 生成一些示例数据
ydata = [57.6 41.9 31.0 22.7 16.6 12.2 8.9 6.5];
x0 = [1, 0]; % 初始参数猜测值
x = lsqcurvefit(fun, x0, xdata, ydata)
```

运行结果为 $a(1)=60.0484$, $a(2)=0.1950$, $a(3)=1.8537$

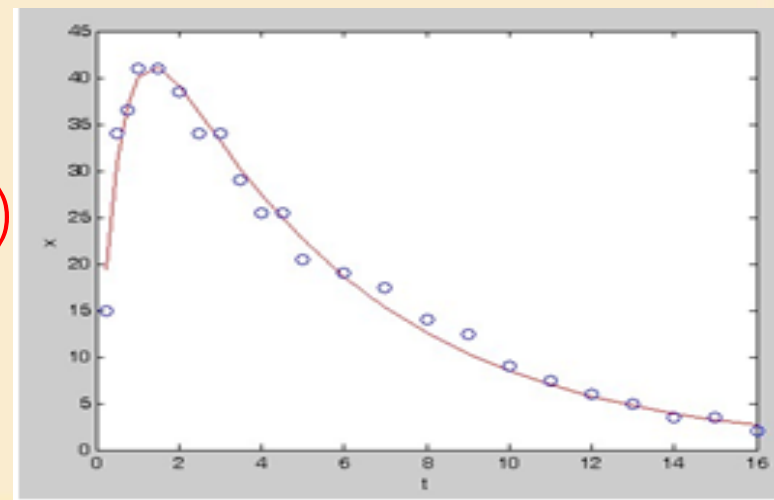
故该饮酒人喝一瓶啤酒后血液中的酒精含量为:

$$x(t) = 60.0484 \times (e^{-0.1950t} - e^{-1.8537t})$$

此时, $k_1=0.1950$, $k_2=1.8537$, $c=510.7809$.

$x(t)$ 的拟合曲线(红线)与原始观测值(蓝色o)图, 程序如下:

```
a=[60.0480,0.1950,1.8537];  
z=a(1)*(exp(-a(2)*x)-exp(-a(3)*x))  
plot(x,z,'r',x,y,'ob');  
xlabel('t');ylabel('x');
```



$$x(t) = 60.0484 \times (e^{-0.1950t} - e^{-1.8537t})$$

当 $t=6$ 时, $x(6)=18.6360(\text{mg}/100\text{ml})$, 因此, 该驾驶人中午12点喝一瓶啤酒, 下午6点检查时符合驾车标准。
若中午第一次喝一瓶啤酒后6个小时, 到晚上再喝时, 胃肠道中剩余酒精量为

$$m(6) = ce^{-6k_1} = 158.5295$$

血液中的酒精含量为

$$x(6) = ck_1 / (k_2 - k_1) \cdot (e^{-6k_1} - e^{-6k_2}) = 18.6360$$

此时再喝一瓶啤酒，设胃肠道中和血液中的酒精含量分别为 $M(t)$ 和 $X(t)$ ，则 t 时刻胃肠道中的酒精含量方程为 $dM/dt = -0.1950M(t)$; $M(0) = m(6) + c = 669.3104$

该微分方程的Matlab程序为：

`dsolve('Dm+0.1950*m=0','m(0)=669.3104','t')`

运行结果为 $\text{ans} = 418319/625 * \exp(-39/200 * t)$

即胃肠道中的酒精含量方程为

$$M(t) = 669.3104e^{-0.1950t}$$

同理， t 时刻血液中的酒精含量方程为

$$dX/dt = k_1 M(t) - k_2 X(t); \quad X(0) = x(6)$$

将 $M(t)$ 及相应的参数代入，得到

$$dX/dt = 130.5155e^{-0.1950t} - 1.8537X(t); \quad X(0) = x(6) = 18.6360$$

其Matlab程序为：

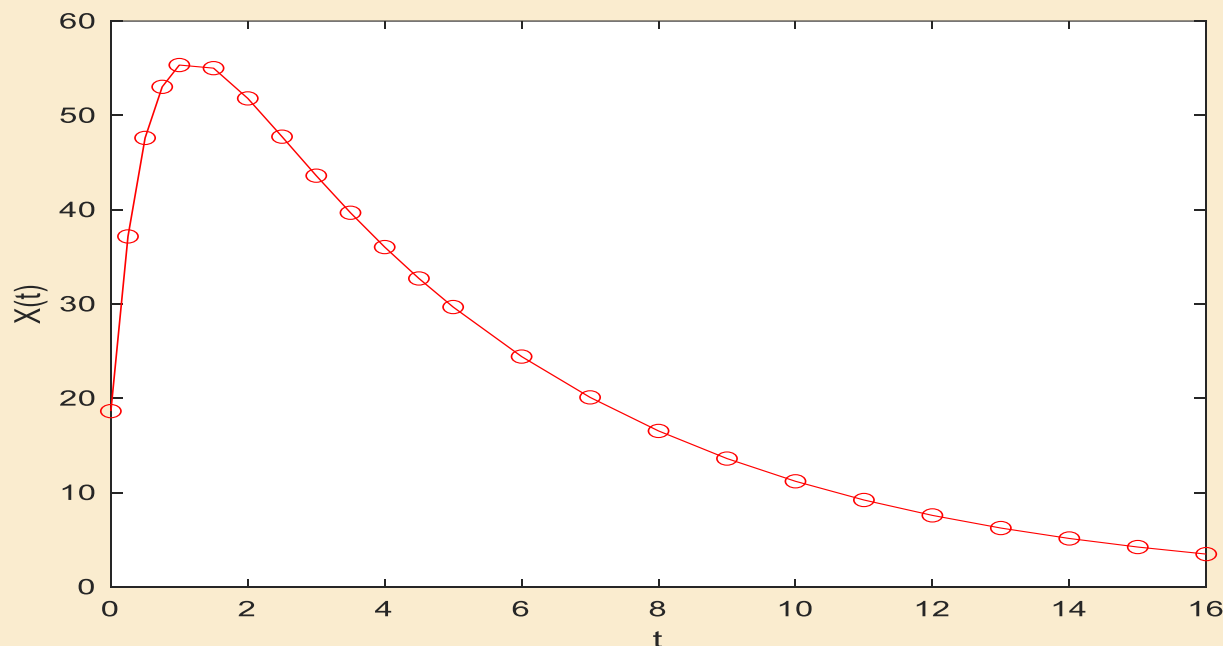
```
dsolve('Dx-130.5155*exp(-  
0.1950*t)+1.8537*x=0','x(0)=18.6360','t')
```

运行结果为ans = 1305155/16587*exp(-39/200*t)-
249009917/4146750*exp(-18537/10000*t)， 即有

$$X(t) = 78.6854e^{-0.1950t} - 60.0494e^{-1.8537t}$$

喝过第二瓶啤酒后血液中的酒精含量方程:

$$X(t) = 78.6854e^{-0.1950t} - 60.0494e^{-1.8537t}$$



$X(6) > 20$, 因此驾驶人若中午12点喝一瓶啤酒, 下午6点又喝一瓶啤酒, 则凌晨零点驾车时血液内酒精含量超过20mg/100ml, 此时若驾车则可定为饮酒驾车。

常微分方程数值解法及其实验

Numerical Methods for Ordinary Differential Equations and Experiments

为什么要学习微分方程数值解

- 微分方程是研究函数变化规律的重要工具，有着广泛的应用。如：

物体的运动， 电路的电压， 人口增长的预测

- 许多微分方程没有解析解，数值解法是求解的重要手段，如

$$\frac{dy}{dx} = y^2 + x,$$

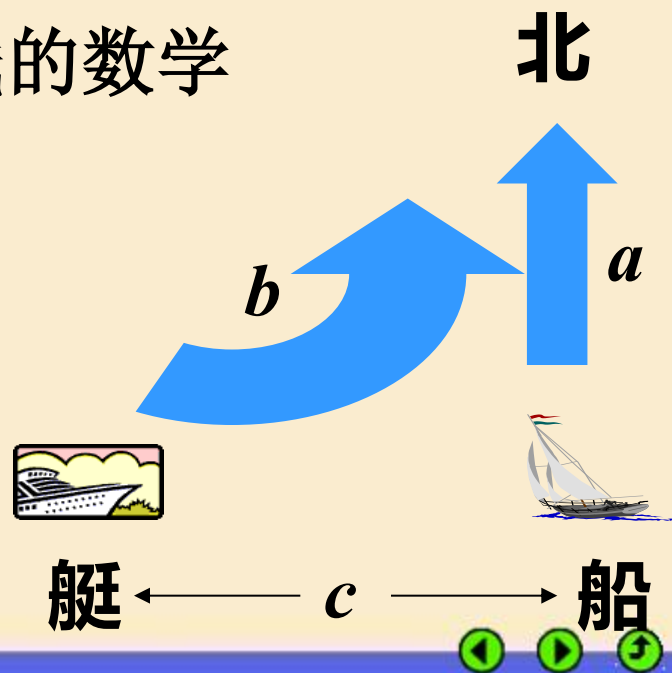
$$\dot{x}(t) = -axy$$

$$\dot{y}(t) = axy - by$$

实例1 海上缉私

海防某部缉私艇上的雷达发现正东方向 c 海里处有一艘走私船正以速度 a 向正北方向行驶，缉私艇立即以最大速度 $b(>a)$ 前往拦截。如果用雷达进行跟踪时，可保持缉私艇的速度方向始终指向走私船。

- 建立任意时刻缉私艇位置及航线的数学模型，并求解；
- 求出缉私艇追上走私船的时间。



实例1 海上缉私

建立坐标系如图: $t=0$ 艇在 $(0, 0)$, 船在 $(c, 0)$; 船速 a , 艇速 b

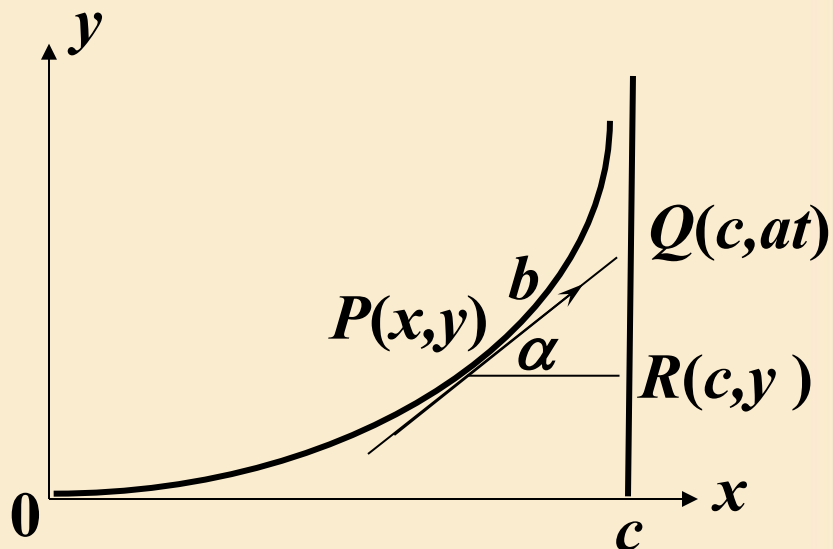
时刻 t 艇位于 $P(x, y)$, 船到达 $Q(c, at)$

模型:

$$\frac{dx}{dt} = b \cos \alpha, \frac{dy}{dt} = b \sin \alpha$$

$$\begin{cases} \frac{dx}{dt} = \frac{b(c-x)}{\sqrt{(c-x)^2 + (at-y)^2}} \\ \frac{dy}{dt} = \frac{b(at-y)}{\sqrt{(c-x)^2 + (at-y)^2}} \end{cases}$$

由方程无法得到 $x(t)$, $y(t)$ 的解析解
需要用数值解法求解



“常微分方程初值问题数值解”的提法

设 $y' = f(x, y)$, $y(x_0) = y_0$ 的解 $y = y(x)$ 存在且唯一

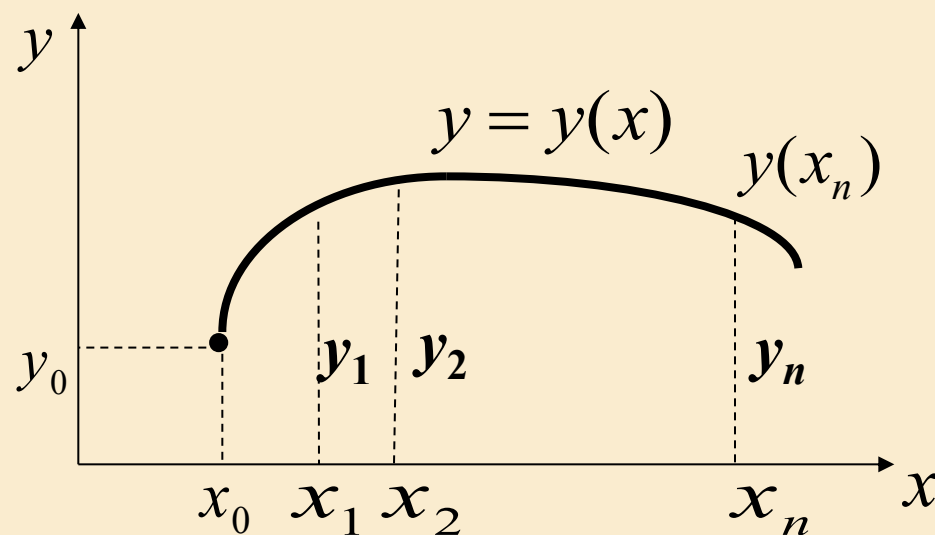
不求解析解 $y = y(x)$ (无解析解或求解困难)

而在一系列离散点 $x_1 < x_2 < \cdots < x_n < \cdots$

求 $y(x_n)$ 的近似值 y_n ($n = 1, 2, \cdots$)

通常取等步长 h

$$x_n = x_0 + nh$$



欧拉方法

$$y' = f(x, y), y(x_0) = y_0$$

向前差分公式

基本思路

在小区间 $[x_n, x_{n+1}]$ 上 $y' = [y(x_{n+1}) - y(x_n)] / h$,
 $f(x, y)$ 中的 x 取 $[x_n, x_{n+1}]$ 内的某一点

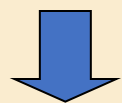
x取不同点



各种欧拉公式

$$y(x_{n+1}) = y(x_n) + hf(x, y(x)), x \in [x_n, x_{n+1}]$$

x 取左端点 x_n



$$y(x_{n+1}) = y(x_n) + hf(x_n, y(x_n))$$

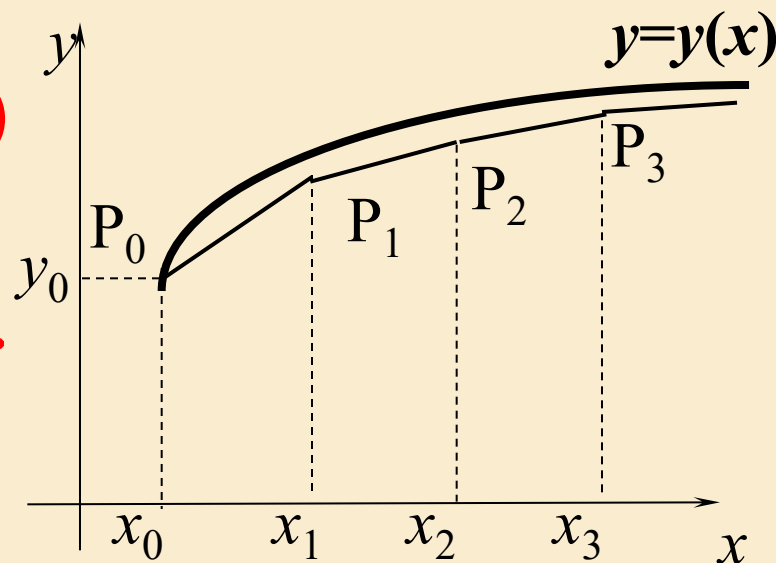
$$y_n \approx y(x_n), y_{n+1} \approx y(x_{n+1})$$



$$y_{n+1} = y_n + hf(x_n, y_n), n = 0, 1, \dots$$

向前欧拉公式

显式公式



欧拉方法

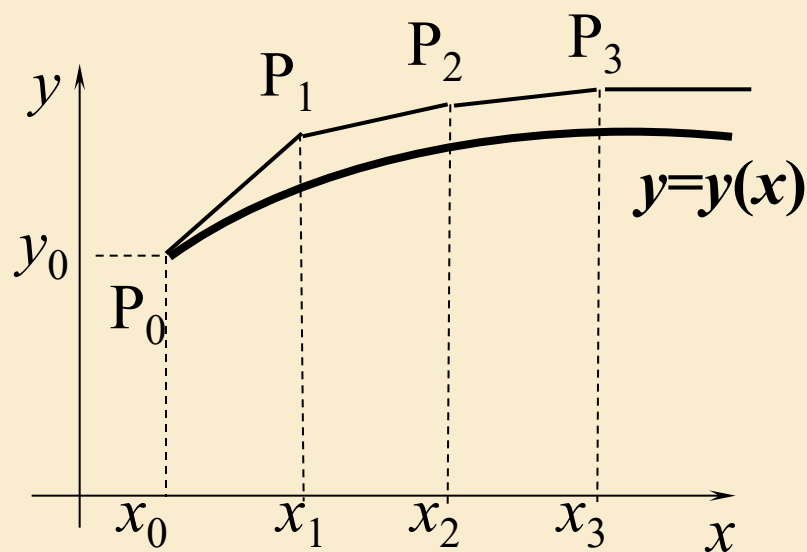
$$y(x_{n+1}) = y(x_n) + hf(x, y(x)), x \in [x_n, x_{n+1}]$$

x 取右端点, $y_n \approx y(x_n), y_{n+1} \approx y(x_{n+1})$ 

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}), n = 0, 1, \dots$$

向后欧拉公式

隐式公式



右端 y_{n+1} 未知, 需迭代求解

$$y(x_{n+1}) = y(x_n) + hf(x_n, y(x_n))$$

$$\text{初值 } y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$$

$$y_{n+1}^{(k+1)} = y_n + hf(x_{n+1}, y_{n+1}^{(k)})$$

$$k = 0, 1, 2, \dots, n = 0, 1, 2, \dots$$

$$\lim_{k \rightarrow \infty} y_{n+1}^{(k)} = y_{n+1}$$

向前欧拉公式

$$y_{n+1} = y_n + hf(x_n, y_n)$$

向后欧拉公式

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$$

二者平均得到梯形公式

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})], n = 0, 1, \dots$$

仍为隐式公式，需迭代求解

改进欧拉公式

将梯形公式的迭代过程简化为两步

$$\bar{y}_{n+1} = y_n + hf(x_n, y_n) \quad \text{预测}$$

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})], n = 0, 1, \dots$$

校正

微分方程数值解法的误差分析

数值解法: 计算微分方程精确解 $y(x_n)$ 的近似值 y_n

按照步长 h 一步步计算, 每步都有误差;

每一步的误差会逐步积累, 称**累积误差**.

讨论计算一步出现的误差

假定 $y_n = y(x_n)$, 估计 y_{n+1} 的误差: $y(x_{n+1}) - y_{n+1}$

局部截断误差

误差分析 估计欧拉公式的局部截断误差

$y(x_{n+1})$ 在 x_n 处作Taylor展开:

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + O(h^3)$$

向前欧拉公式 $y_{n+1} = y_n + hf(x_n, y_n)$

$$y_n = y(x_n)$$



$$y' = f(x, y), y(x_0) = y_0$$

$$y_{n+1} = y(x_n) + hf(x_n, y(x_n)) = y(x_n) + hy'(x_n)$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1} = \frac{h^2}{2} y''(x_n) + O(h^3) = O(h^2)$$

局部截断误差主项为 $\frac{h^2}{2} y''(x_n)$

误差分析 估计欧拉公式的局部截断误差

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + O(h^3)$$

向后欧拉公式 $y_{n+1} = y_n + hf(x_{n+1}, y_{n+1})$



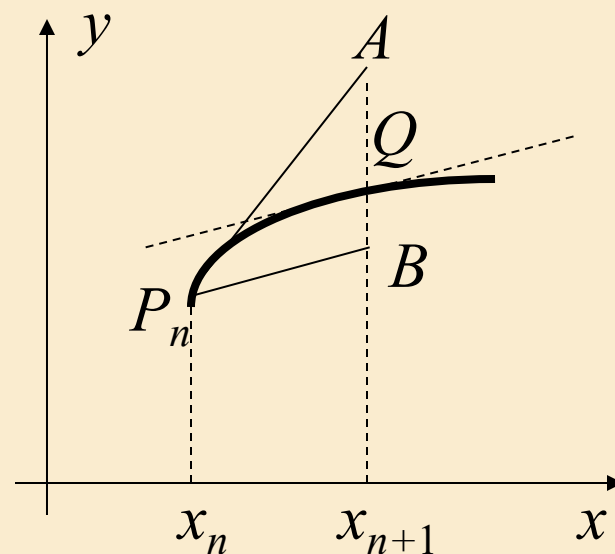
$$T_{n+1} = y(x_{n+1}) - y_{n+1} = -\frac{h^2}{2} y''(x_n) + O(h^3) = O(h^2)$$

局部截断误差主项为 $-\frac{h^2}{2} y''(x_n)$

**梯形
公式**

向前、向后欧拉公式的平均

$$T_{n+1} = -\frac{h^3}{12} y'''(x_n) + O(h^4) = O(h^3)$$



误差分析

算法精度的阶的定义

一个算法的局部截断误差为 $O(h^{p+1})$



该算法具有 p 阶精度

	局部截断误差	精度
向前欧拉公式	$O(h^2)$	1阶
向后欧拉公式	$O(h^2)$	1阶
梯形公式	$O(h^3)$	2阶
改进欧拉公式	$O(h^3)$	2阶
经典龙格-库塔公式	$O(h^5)$	4阶

龙格-库塔方法

$$y(x_{n+1}) = y(x_n) + hf(x, y(x)), x \in [x_n, x_{n+1}]$$

- 向前，向后欧拉公式：
用 $[x_n, x_{n+1}]$ 内 1 个点的导数代替 $f(x, y(x))$
- 梯形公式，改进欧拉公式：
用 $[x_n, x_{n+1}]$ 内 2 个点导数的平均值代替 $f(x, y(x))$

龙格-库塔方法的基本思想

在 $[x_n, x_{n+1}]$ 内多取几个点，将它们的导数加权平均代替 $f(x, y(x))$ ，设法构造出精度更高的计算公式。

取区间 $[x_n, x_{n+1}]$ 上的两个导数, 推导2阶龙格-库塔公式的过程。

在 x_n 和 $x_n + \alpha h (0 < \alpha < 1)$ 两个点上取导数作线性组合

$$\begin{cases} y_{n+1} = y_n + h(\lambda_1 k_1 + \lambda_2 k_2), \\ k_1 = f(x_n, y_n), \\ k_2 = f(x_n + \alpha h, y_n + \alpha h k_1), \quad 0 < \alpha \leq 1. \end{cases}$$

对 k_2 作二元泰勒展开, 且利用 $k_1 = y' = f, y'' = f_x + f \cdot f_y$,

$$\begin{cases} y_{n+1} = y(x_n) + h(\lambda_1 k_1 + \lambda_2 k_2), \\ k_1 = f(x_n, y(x_n)) = y'(x_n), \\ k_2 = f(x_n + \alpha h, y(x_n) + \alpha h k_1) \\ \quad = y'(x_n) + \underbrace{\alpha h f_x(x_n, y(x_n)) + \alpha h k_1 f_y(x_n, y(x_n))}_{\alpha h y''(x_n)} + O(h^2) \\ \quad = y'(x_n) + \alpha h (f_x + f \cdot f_y) + O(h^2) = y'(x_n) + \alpha h y''(x_n) + O(h^2). \end{cases}$$

于是, $y_{n+1} = y(x_n) + (\lambda_1 + \lambda_2) h y'(x_n) + \lambda_2 \alpha h^2 y''(x_n) + O(h^3)$.

$$y_{n+1} = y(x_n) + (\lambda_1 + \lambda_2)hy'(x_n) + \lambda_2\alpha h^2 y''(x_n) + O(h^3).$$

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + O(h^3) \quad \text{精确解}$$

$$T_{n+1} = y(x_{n+1}) - y_{n+1}$$

$$= (1 - \lambda_1 - \lambda_2)hy'(x_n) + \left(\frac{1}{2} - \lambda_2\alpha\right)h^2 y''(x_n) + O(h^3).$$

只需选取 $\lambda_1, \lambda_2, \alpha$ 满足

$$\lambda_1 + \lambda_2 = 1, \quad \lambda_2\alpha = \frac{1}{2},$$

$$\begin{cases} y_{n+1} = y_n + h(\lambda_1 k_1 + \lambda_2 k_2), \\ k_1 = f(x_n, y_n), \\ k_2 = f(x_n + \alpha h, y_n + \alpha h k_1), \quad 0 < \alpha \leq 1. \end{cases}$$

具有 2 阶的精度.

龙格-库塔
方法的一
般形式

$$\begin{cases} y_{n+1} = y_n + h \sum_{i=1}^L \lambda_i k_i \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + c_2 h, y_n + c_2 h k_1) \\ \dots \\ k_i = f(x_n + c_i h, y_n + c_i h \sum_{j=1}^{i-1} a_{ij} k_j), \quad i = 3, 4, \dots, L \end{cases}$$

L级L阶

$$\lambda_i, c_i, a_{ij} \text{ 满足 } \sum_{i=1}^L \lambda_i = 1, 0 \leq c_i \leq 1, \sum_{j=1}^{i-1} a_{ij} = 1$$

使精度尽量高

常用的(经典)
龙格—库塔
公式

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 = f(x_n, y_n) \\ k_2 = f(x_n + h/2, y_n + h k_1 / 2) \\ k_3 = f(x_n + h/2, y_n + h k_2 / 2) \\ k_4 = f(x_n + h, y_n + h k_3) \end{cases}$$

4级4阶

不足:收敛速度较慢

微分方程组和高阶方程初值问题的数值解

欧拉方法和龙格-库塔方法可直接推广到微分方程组

$$\begin{cases} y' = f(x, y, z) \\ z' = g(x, y, z) \\ y(x_0) = y_0, z(x_0) = z_0 \end{cases} \xrightarrow{\text{向前欧拉公式}} \begin{cases} y_{n+1} = y_n + hf(x_n, y_n, z_n) \\ z_{n+1} = z_n + hg(x_n, y_n, z_n) \\ n = 0, 1, 2, \dots \end{cases}$$

高阶方程需要先降阶为一阶微分方程组

$$y'' = f(x, y, y') \xrightarrow{y_1 = y} \begin{cases} y_1' = y_2 \\ y_2' = f(x, y_1, y_2) \end{cases}$$

龙格—库塔方法的 MATLAB 实现

$$\dot{x}(t) = f(t, x), x(t_0) = x_0, x = (x_1, \cdots, x_n)^T, f = (f_1, \cdots, f_n)^T$$

`[t,x]=ode23(@f,ts,x0,opt)` 3级2阶龙格-库塔公式

`[t,x]=ode45(@f,ts,x0,opt)` 5级4阶龙格-库塔公式

f是待解方程写成
的函数m文件:

`function dx=f(t,x)` %以列向量形
`dx=[f1; f2;...; fn]` 式表示方程

`ts = [t0,t1, ...,tf]` 输出指定时刻 `t0,t1, ...,tf` 的函数值

`ts = t0:k:tf` 输出 `[t0,tf]` 内等分点处的函数值

`x0`为函数初值(n维) 输出`t=ts`, `x`为相应函数值(n维)

`opt`为选项, 缺省时精度为: 相对误差 10^{-3} ,
绝对误差 10^{-6} , 计算步长按精度要求自动调整

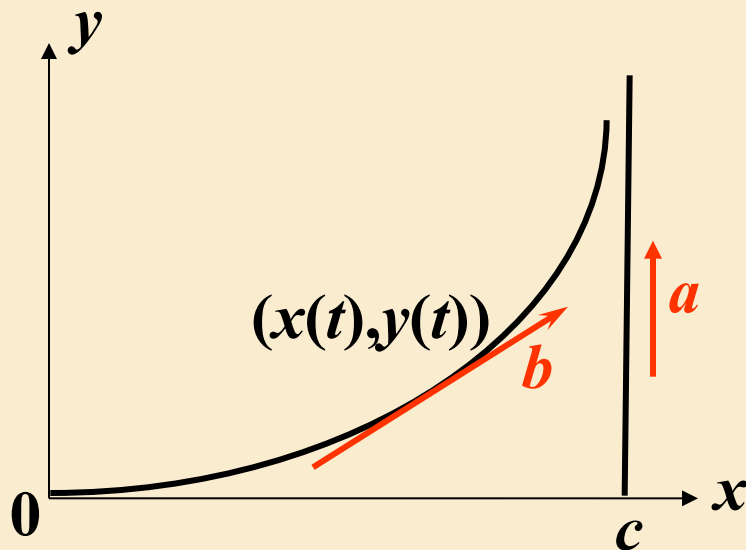
实例1 海上缉私 (续)

模型的数值解

$$\begin{cases} \frac{dx}{dt} = \frac{b(c-x)}{\sqrt{(c-x)^2 + (at-y)^2}} \\ \frac{dy}{dt} = \frac{b(at-y)}{\sqrt{(c-x)^2 + (at-y)^2}} \end{cases}$$

$$x(0) = 0, \quad y(0) = 0$$

设: 船速 $a=20$ (海里/小时)
艇速 $b=40$ (海里/小时)
距离 $c=15$ (海里)



求: 缉私艇的位置 $x(t), y(t)$
缉私艇的航线 $y(x)$

实例1 海上缉私(续)

模型的数值解

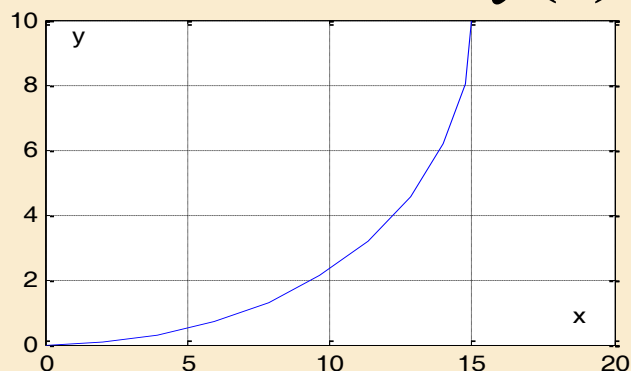
$$a=20, b=40, c=15$$

走私船的位置

$$x_1(t) = c = 15$$

$$y_1(t) = at = 20t$$

缉私艇的航线 $y(x)$



$t=0.5$ 时缉私艇追上走私船

t	$x(t)$	$y(t)$	$y_1(t)$
0	0	0	0
0.05	1.9984	0.0698	1.0
0.10	3.9854	0.2924	2.0
0.15	5.9445	0.6906	3.0
0.20	7.8515	1.2899	4.0
0.25	9.6705	2.1178	5.0
0.30	11.3496	3.2005	6.0
0.35	12.8170	4.5552	7.0
0.40	13.9806	6.1773	8.0
0.45	14.7451	8.0273	9.0
0.50	15.0046	9.9979	10.0

模型的数值解

实例1

海上缉私 (续)

设 b , c 不变, a 变大
为30, 35, ...接近40,
观察解的变化:

$$a=35, b=40, c=15$$

$t=?$ 缉私艇追上走私船

累积误差较大

提高精度!

t	$x(t)$	$y(t)$	$y_1(t)$
0	0	0	0
0.1	3.9561	0.5058	3.5
0.2	7.5928	2.1308	7.0
0.3	10.5240	4.8283	10.5
0.4	12.5384	8.2755	14.0
0.5	13.7551	12.0830	17.5
...	...	...	...
1.2	14.9986	40.0164	42.0
1.3	14.9996	44.0165	45.5
1.4	15.0117	48.0183	49.0
1.5	15.0023	52.0146	52.5
1.6	14.9866	55.9486	56.0

实例1

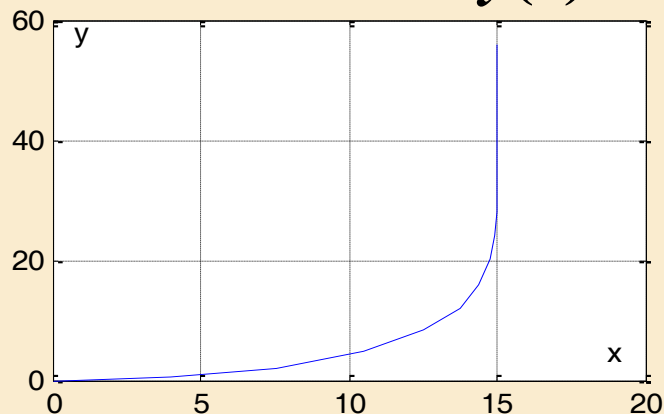
海上缉私 (续)

```
opt=odeset('RelTol',1e-6,
            'AbsTol',1e-9);
```

```
[t,x]=ode45(@jisi,ts,x0,opt);
```

$a=35, b=40, c=15$

缉私艇的航线 $y(x)$



$t=1.6$ 时缉私艇追上走私船

判断“追上”的有效方法?

模型的数值解

t	$x(t)$	$y(t)$	$y_1(t)$
0	0	0	0
0.1	3.956104	0.505813	3.5
0.2	7.592822	2.130678	7.0
0.3	10.521921	4.829308	10.5
0.4	12.539454	8.269840	14.0
0.5	13.753974	12.075344	17.5
...	...	...	...
1.2	14.999616	40.000005	42.0
1.3	14.999963	44.000005	45.5
1.4	14.999993	48.000005	49.0
1.5	14.999998	52.000005	52.5
1.6	15.000020	55.999931	56.0

实例1 海上缉私(续)

模型的解析解

$$\frac{dx}{dt} = \frac{b(c-x)}{\sqrt{(c-x)^2 + (at-y)^2}}, \quad \frac{dy}{dt} = \frac{b(at-y)}{\sqrt{(c-x)^2 + (at-y)^2}}$$

$$\Rightarrow \frac{dy}{dx} = \frac{at-y}{c-x} \quad \Rightarrow (c-x) \frac{dy}{dx} + y = at \quad \Rightarrow (c-x) \frac{d^2 y}{dx^2} = a \frac{dt}{dx}$$

$$\frac{ds}{dt} = b$$

$$\frac{dt}{dx} = \frac{dt}{ds} \frac{ds}{dy}$$

$$(c-x) \frac{d^2 y}{dx^2} = \frac{a}{b} \sqrt{1 + \left(\frac{dy}{dx}\right)^2}$$

$$ds = \sqrt{(dx)^2 + (dy)^2}$$

$$\text{令 } p = \frac{dy}{dx}$$

$$(c-x) \frac{dp}{dx} = k \sqrt{1+p^2}, \quad k = \frac{a}{b}$$

$$\frac{dy}{dx} = \frac{dy}{dt} \frac{dt}{dx}$$

$$p(0) = 0$$

实例1 海上缉私(续)

模型的解析解

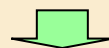
$$(c-x) \frac{dp}{dx} = k\sqrt{1+p^2}$$

$$p(0) = 0$$

$$\sqrt{1+p^2} + p = \left(\frac{c-x}{c}\right)^{-k}$$

$$\sqrt{1+p^2} - p = \left(\frac{c-x}{c}\right)^k$$

$$p = \frac{1}{2} \left[\left(\frac{c-x}{c}\right)^{-k} - \left(\frac{c-x}{c}\right)^k \right]$$



$$k = a/b < 1$$

$$y(0) = 0$$



$$y = \frac{c}{2} \left[\frac{1}{1+k} \left(\frac{c-x}{c}\right)^{1+k} - \frac{1}{1-k} \left(\frac{c-x}{c}\right)^{1-k} \right] + \frac{kc}{1-k^2}$$

缉私艇的航线 $y(x)$ 的解析解

$x=c$ 时

$$y = \frac{kc}{1-k^2} = \frac{abc}{b^2-a^2}$$

缉私艇追上走私船的 y 坐标

缉私艇追上走私船的时间:

$$t_1 = \frac{bc}{b^2-a^2}$$

$$a=20, b=40, c=15 \rightarrow t_1=0.5$$

$$a=35, b=40, c=15 \rightarrow t_1=1.6$$

作业：海上缉私问题的进一步研究：

对以下三种情况建立缉私艇位置和航线的数学模型，自己设定速度等参数，求数值解。

- (1) 若走私船沿着与正东方向成某一角度的直线行驶；
- (2) 若走私船仍向正北方向行驶，一段时间（设尚未被缉私船追上）后又掉头返回；
- (3) 讨论走私船和缉私艇位于任意初始位置，走私船按任意给定的路线行驶的情况。